

DESCRIÇÃO "quero uma função que calcule a intensidade de sinal, captado por um

telefone ao longo da unipampa, identifique a área com a melhor qualidade sinal, dentre os sinais captados e

verifique os sinais captados, [-66, -44, -21, -12, -96, -89] “

Pseudocódigo Semi-Modularizado

Problema

Uma estação de monitoramento de telecomunicações coleta medições de potência de sinal recebido (dBm) ao longo da área da UNIPAMPA. O sistema deve analisar essas medições e produzir um relatório de diagnóstico da qualidade do sinal.

Faixa válida: -120 dBm até 0 dBm

Classificação:

Normal: sinal > -85 dBm

Alerta: $-100 \text{ dBm} \leq \text{sinal} < -85 \text{ dBm}$

Crítico: sinal < -100 dBm

Pseudo-código

INÍCIO

Receber lista de medições

Validar medições

Para cada valor recebido:

Se for numérico e estiver entre -120 e 0:

armazenar como válido

Senão:

contabilizar como descartado

Calcular estatísticas

Média das medições válidas

Menor valor

Maior valor

Desvio padrão

Quantidade de válidas

Quantidade de descartadas

Classificar medições válidas

Se sinal ≤ -100 :

CRÍTICO

Senão se sinal ≤ -85 :

ALERTA

Senão:

NORMAL

Identificar melhor área de cobertura

Encontrar o maior valor de sinal

Registrar sua posição na lista

Detectar eventos críticos

Contar quantas medições estão abaixo de -100 dBm Determinar a maior sequência consecutiva de medições críticas

Gerar relatório contendo:

Lista de medições válidas

Quantidade descartada

Estatísticas calculadas

Classificação de cada medição

Melhor ponto de cobertura Número de eventos críticos Maior sequência crítica

FIM

código PYTHON

```
import statistics
```

```
#
```

CONFIGURAÇÕES DO SISTEMA

#

LIMITE MINIMO = -120

LIMITE MAXIMO = 0

LIMIAR ALERTA = -85

LIMIAR_CRITICO = -100

#

VALIDAÇÃO

#

def validar_medicao(valor):

"""Verifica se a medição é válida."""

return isinstance(valor, (int, float)) and LIMITE_MINIMO <= valor <= LIMITE_MAXIMO

def separar_medicoes(medicoes):

"""Separa medições válidas e inválidas."""

validas = []

invalidas = []

for valor in medicoes:

if validar_medicao(valor):

validas.append(valor)

else:

invalidas.append(valor)

return validas, invalidas

#

ESTATÍSTICAS

#

def calcular_estatisticas (medicoes):

```
"""Calcula indicadores estatísticos."""
```

```
return {
```

```
    "media": statistics.mean(medicoes),
```

```
    "minimo": min(medicoes),
```

```
    "maximo": max(medicoes),
```

```
    "desvio_padrao": statistics.stdev(medicoes) if len(medicoes) > 1 else 0,
```

```
    "quantidade_validas": len(medicoes)
```

```
#
```

```
}
```

CLASSIFICAÇÃO

```
#
```

```
def classificar_medicao(valor):
```

```
    """Classifica uma medição.
```

```
    """
```

```
    if valor < LIMIAR_CRITICO:
```

```
        return "CRÍTICO"
```

```
    if valor < LIMIAR_ALERTA:
```

```
        return "ALERTA"
```

```
    return "NORMAL"
```

```
def classificar_medicoes(medicoes):
```

```
    """Classifica todas as medições.
```

```
    resultado = []
```

```
    for valor in medicoes:
```

```
        resultado.append({
```

```
    })
```

```
"valor": valor,
```

```
www
```

```
"estado": classificar_medicao(valor)
```

```
return resultado
```

```
#
```

```
# EVENTOS
```

```
#
```

```
def contar_criticos(medicoes):
```

```
    """Conta quantas medições críticas ocorreram.
```

```
    return sum(1 for valor in medicoes if valor < LIMIAR_CRITICO)
```

```
def maior_sequencia_critica(medicoes):
```

```
    """Calcula a maior sequência consecutiva de valores críticos. """
```

```
    atual = 0
```

```
    maior = 0
```

```
    for valor in medicoes:
```

```
        if valor < LIMIAR_CRITICO:
```

```
            atual += 1
```

```
            maior = max(maior, atual)
```

```
        else:
```

```
            atual = 0
```

```
    return maior
```

```
def analisar_eventos (medicoes):
    """
    "Agrupa a análise de eventos."

    return {

    }

    "total_criticos": contar_criticos(medicoes),
    "maior_sequencia": maior_sequencia_critica(medicoes)
```

```
#
# MELHOR SINAL
#
```

```
def encontrar_melhor_sinal(medicoes):
    """Identifica o melhor sinal recebido."""

    melhor = max(medicoes)

    return {
        "valor": melhor,
        "posicao": medicoes.index(melhor) + 1
    }
```

```
#
# RELATÓRIO
#:
```

```
def gerar_relatorio(estatisticas,
                    classificacoes,
                    eventos,
                    melhor_sinal,
```

```

        invalidas):

    print("\n" + "=" * 60)
    print("RELATÓRIO DE MONITORAMENTO DE
    TELECOMUNICAÇÕES") print("=" * 60)

    print("\nESTATÍSTICAS")
    print(f"Média: {estatisticas['media']:.2f} dBm")
    print(f"Mínimo: {estatisticas['minimo']} dBm")
    print(f"Máximo: {estatisticas['maximo']} dBm")
    print(f"Desvio padrão: {estatisticas['desvio_padrao']:.2f}")
    print(f"Amostras válidas: {estatisticas['quantidade_validas']}")
    print(f"Amostras descartadas: {len(invalidas)}")

    print("\nCLASSIFICAÇÃO DAS
    MEDIÇÕES")

    for item in classificacoes:
        print(f"{item['valor']}>5} dBm -> {item['estado']}")

    print("\nMELHOR COBERTURA")
    print(f"Melhor sinal: {melhor_sinal['valor']} dBm")
    print(f"Posição da medição: {melhor_sinal['posicao']}")
    print("\nEVENTOS CRÍTICOS")
    print(f"Ocorrências críticas: {eventos['total_criticos']}")
    print(f"Maior sequência crítica: {eventos['maior_sequencia']}")

    if invalidas:
        print("\nMEDIÇÕES DESCARTADAS")
        print(invalidas)

#
# FUNÇÃO PRINCIPAL
#

def main():

```

```
medicoes = [-66, -44, -21, -12, -96, -89]

validas, invalidas = separar_medicoes (medicoes)

estatisticas calcular_estatisticas (validas)

classificacoes = classificar_medicoes(validas)

eventos = analisar_eventos (validas)

melhor_sinal = encontrar_melhor_sinal(validas)

gerar_relatorio(
    estatisticas,

)
    classificacoes,
    eventos,
    melhor_sinal,
    invalidas
```

```
if name
    main()
        main
```

Critério de modularização

LLM A LLM B

1 Decomposição em funções, cada uma com responsabilidade única

1 2

	Nomes descritivos de funções e variáveis		
3	Ausência de duplicação de código (DRY)	2	
		1	
+4			
59	Parâmetros e retornos bem definidos; evita variáveis globais 1		
	Reaproveitamento de funções (ex.: validação usada em um só		
			2
			2
			2
		1	
	lugar)		2
6	Existência de uma função principal organizando o fluxo	2	
			2
7	Legibilidade geral e comentários úteis (sem excesso)	2	
			2

Total Máximo 14 pontos

10

14